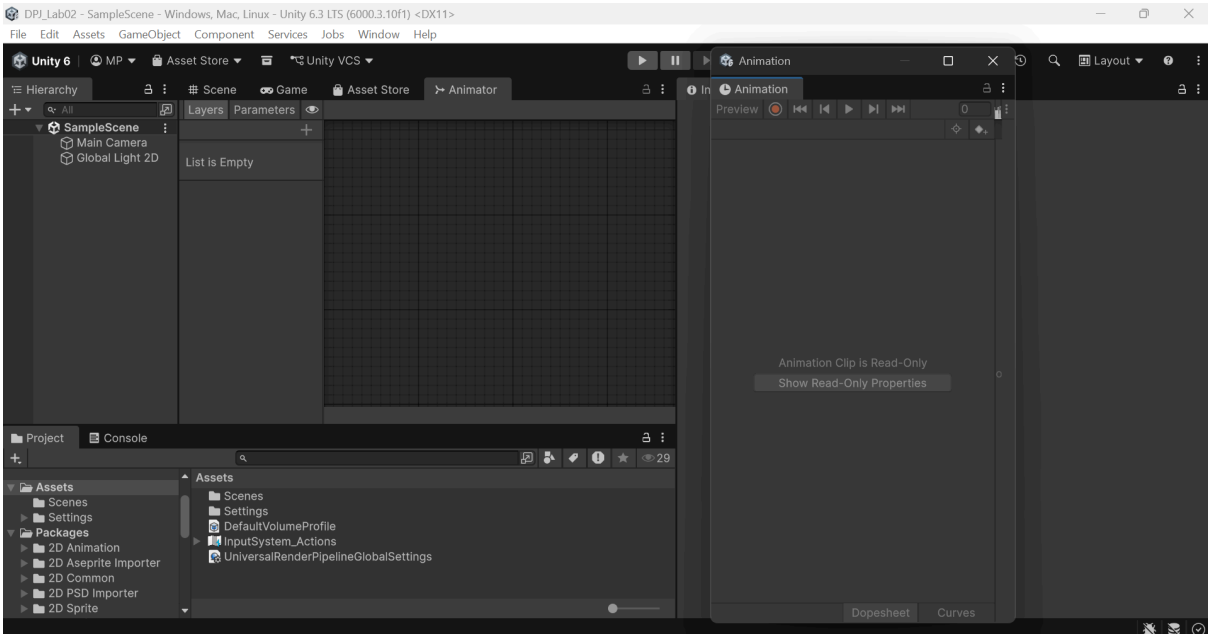
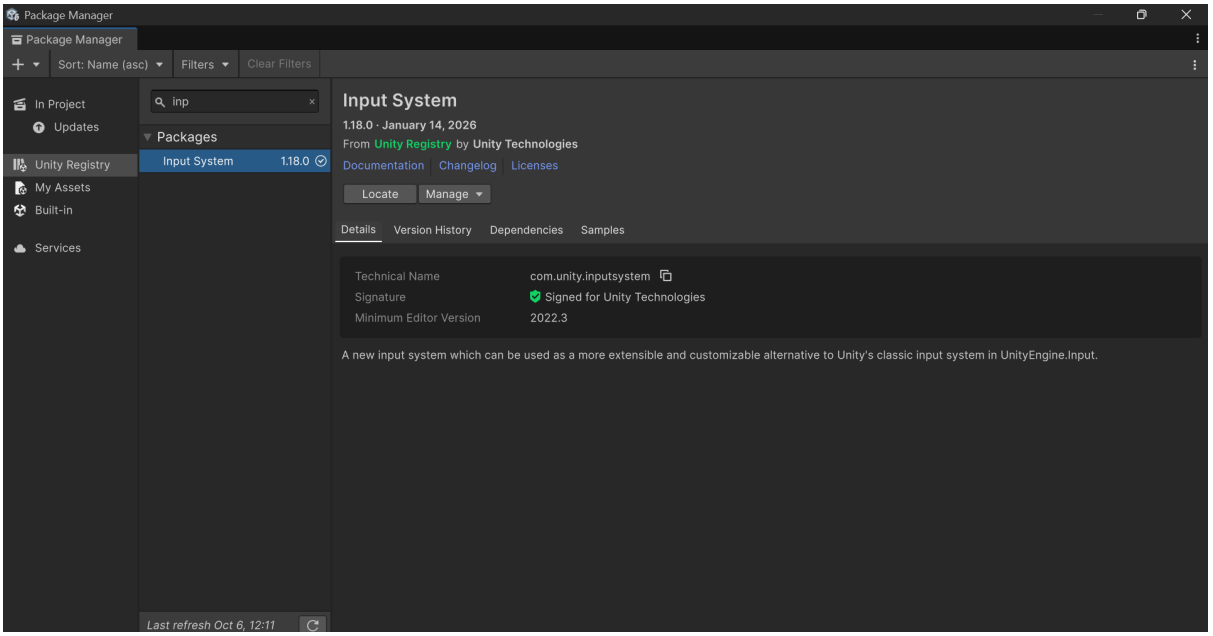
Lab02 DPJ Technical Document

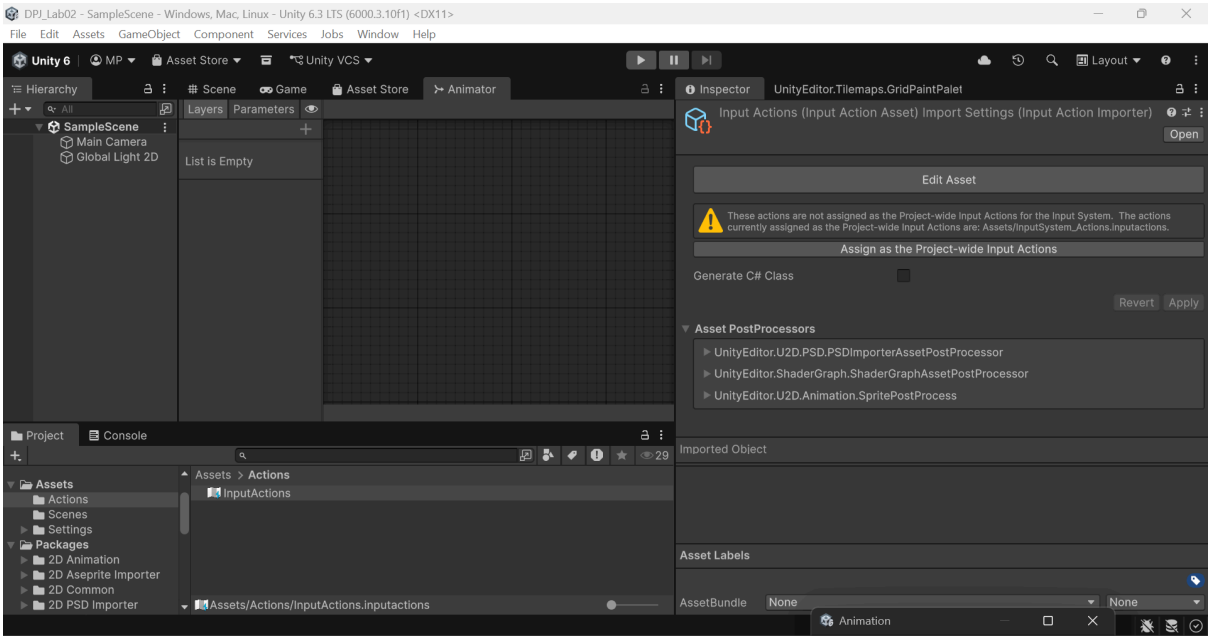
Level 1



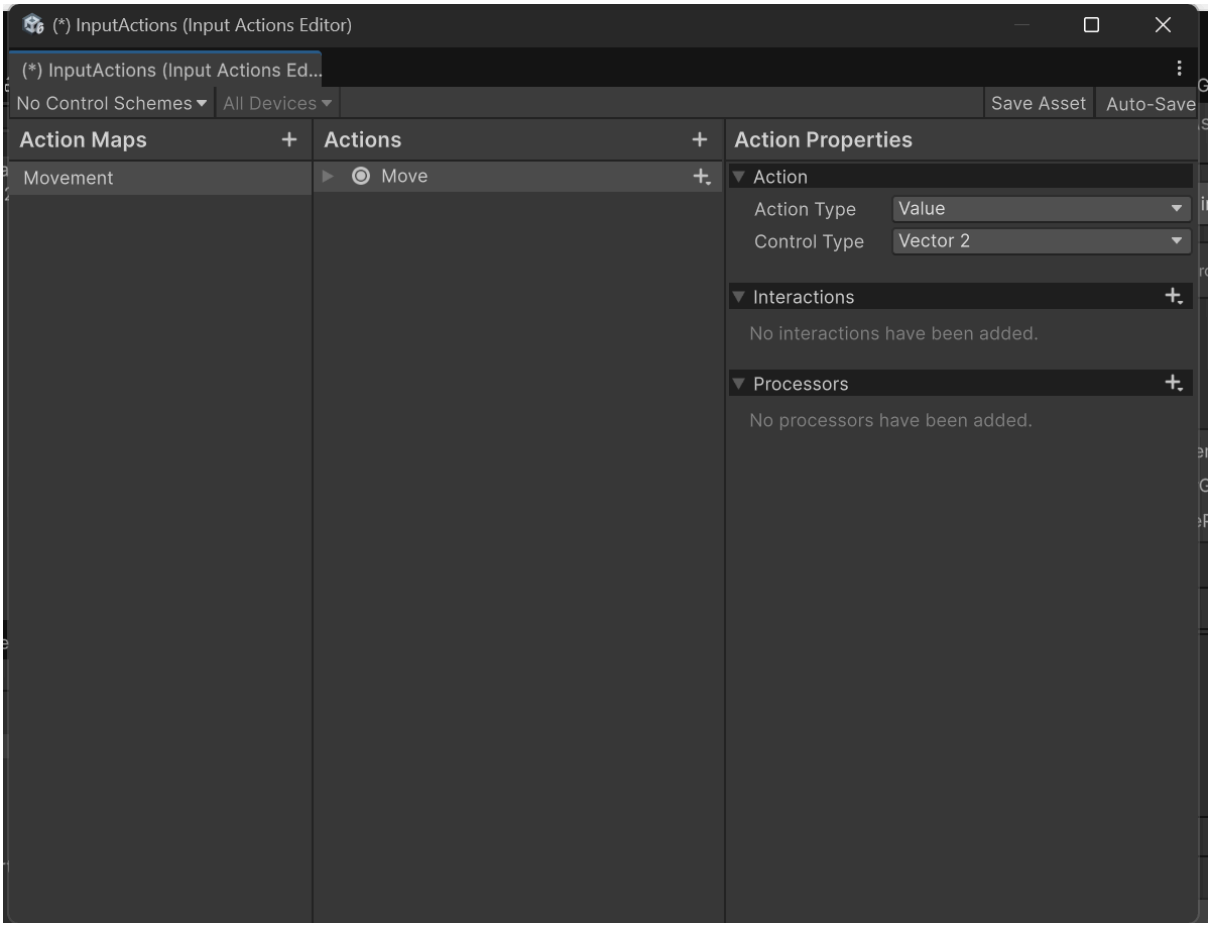
First of all we created a new 2D project called “DPJ_Lab02”, then we opened the animator.



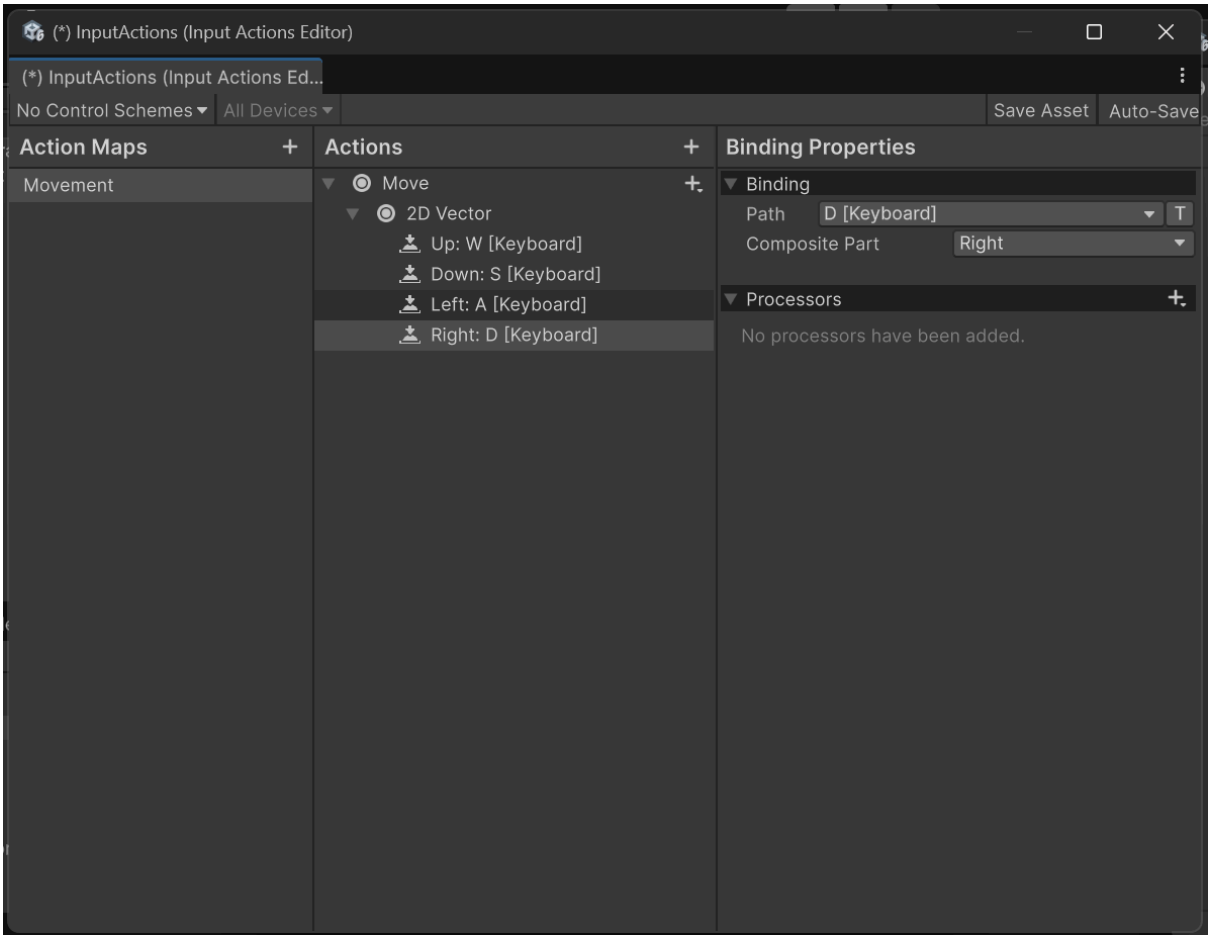
Then we went to install “input system”.

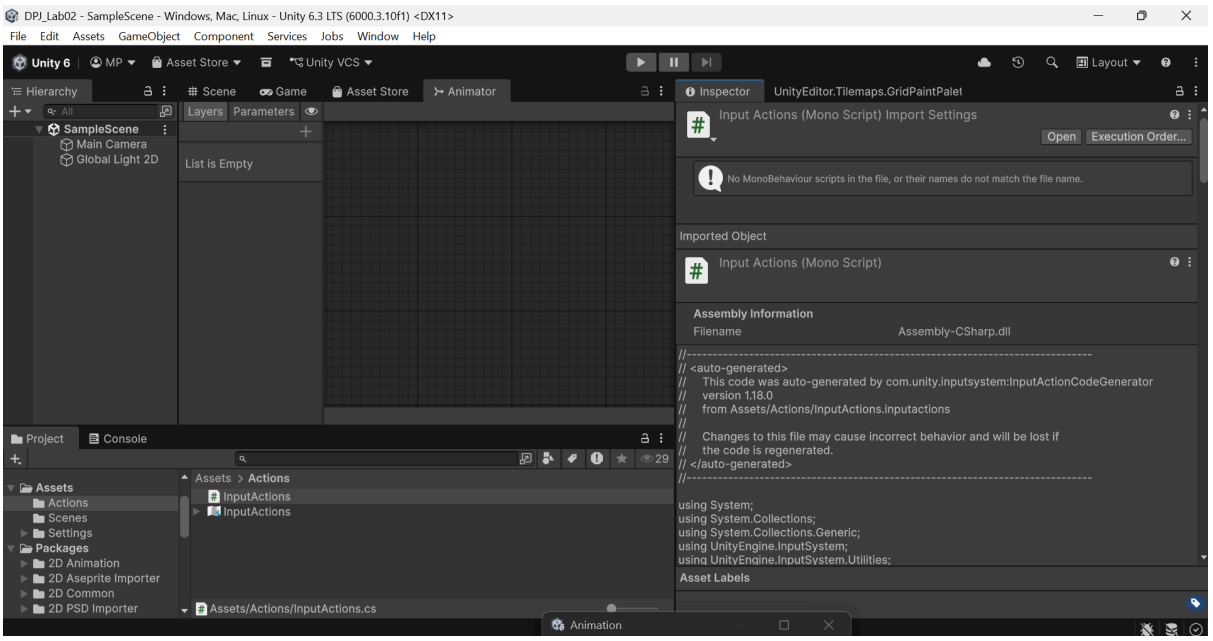


Next inside the assets folder inside of it we created the actions folder.



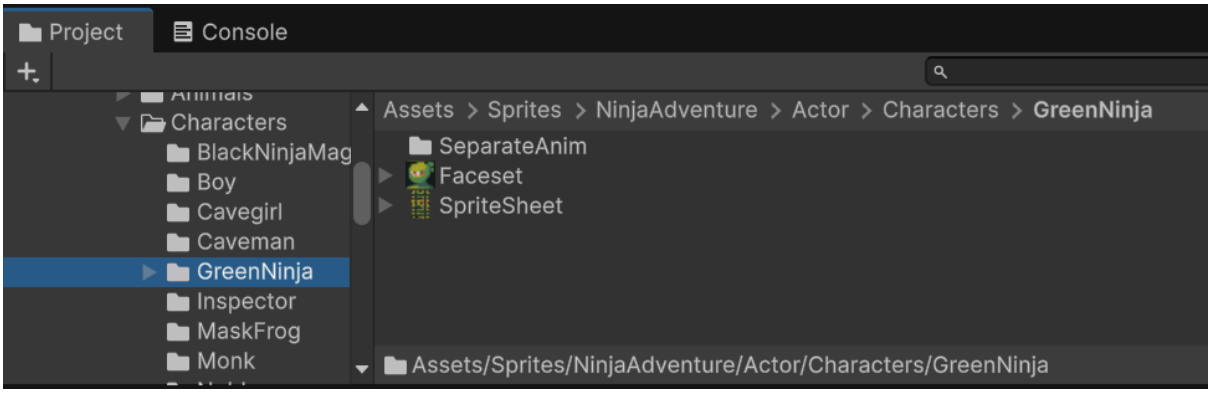
We also then created inside of that folder an input actions resource

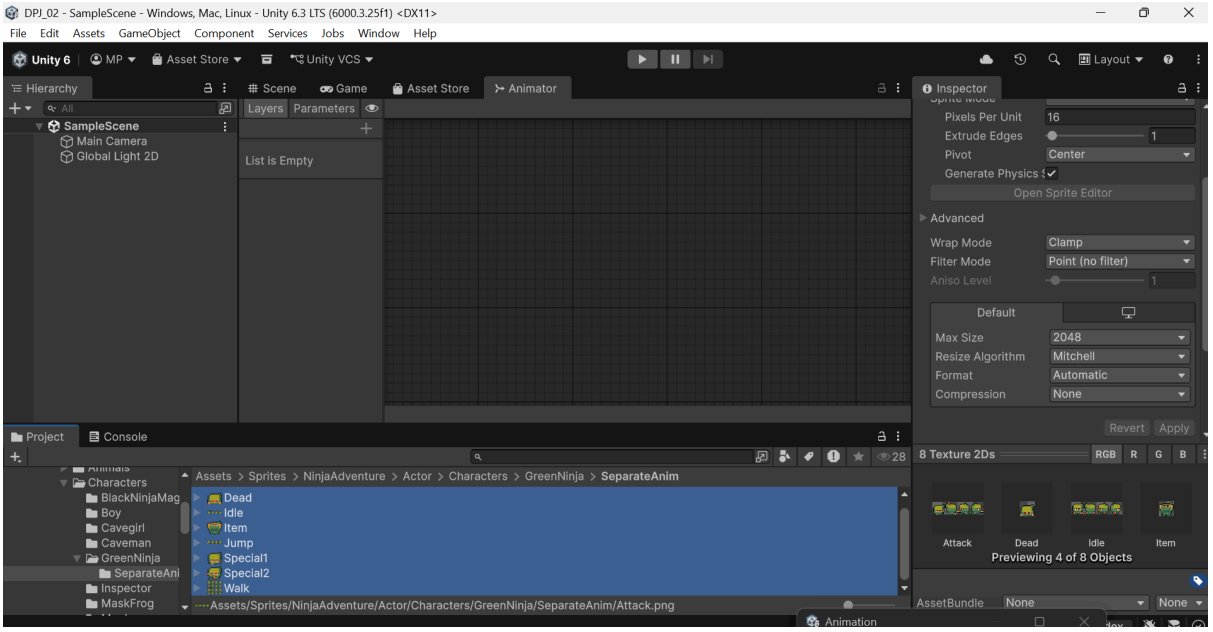




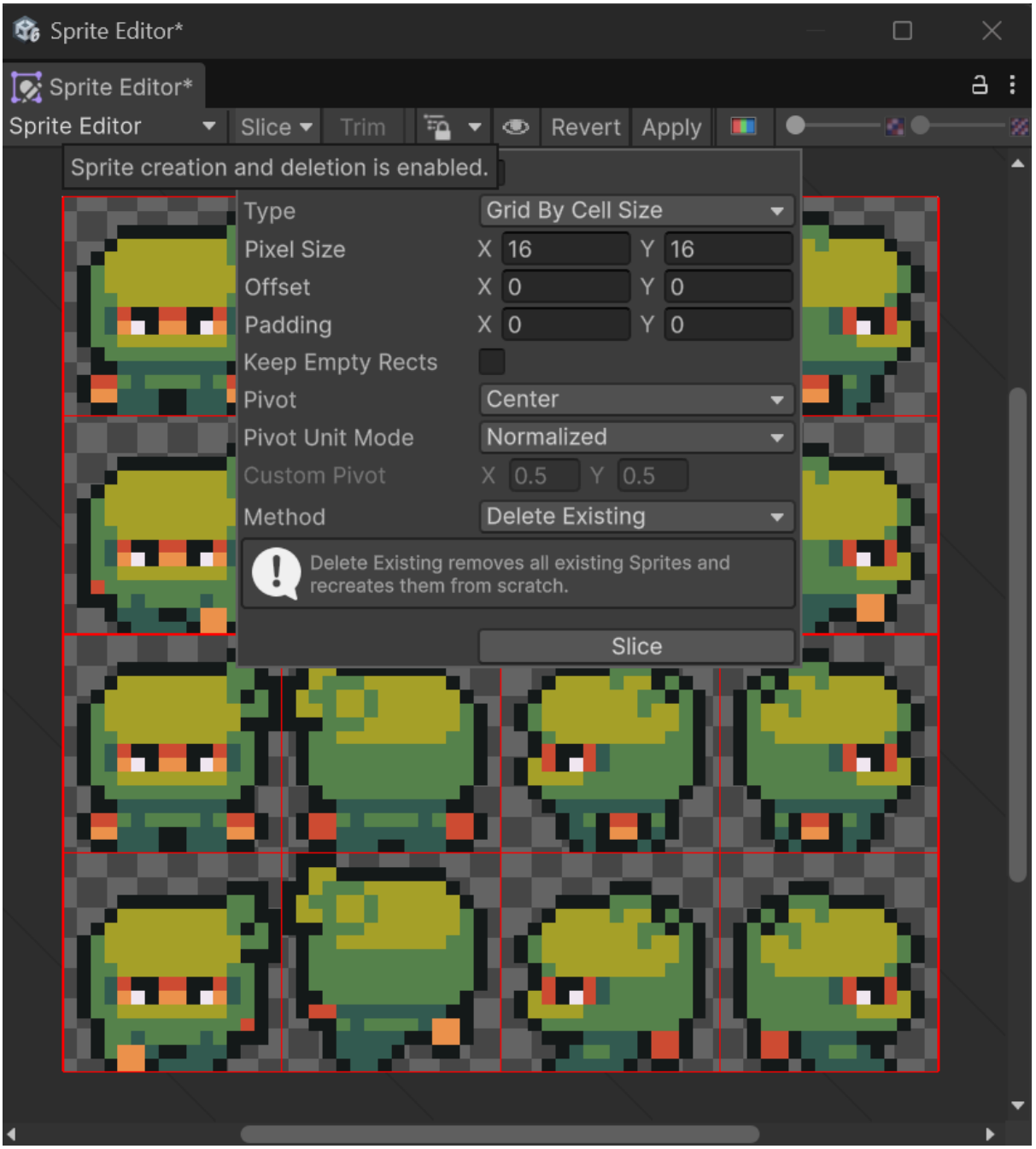
Here on this two images on top we show that we created the action “move” and a control type “vector 2”, inside the “move” action we created the “up”, “down”, “right” and “left” the for each element we assigned a key creating a C# script.

Level 2



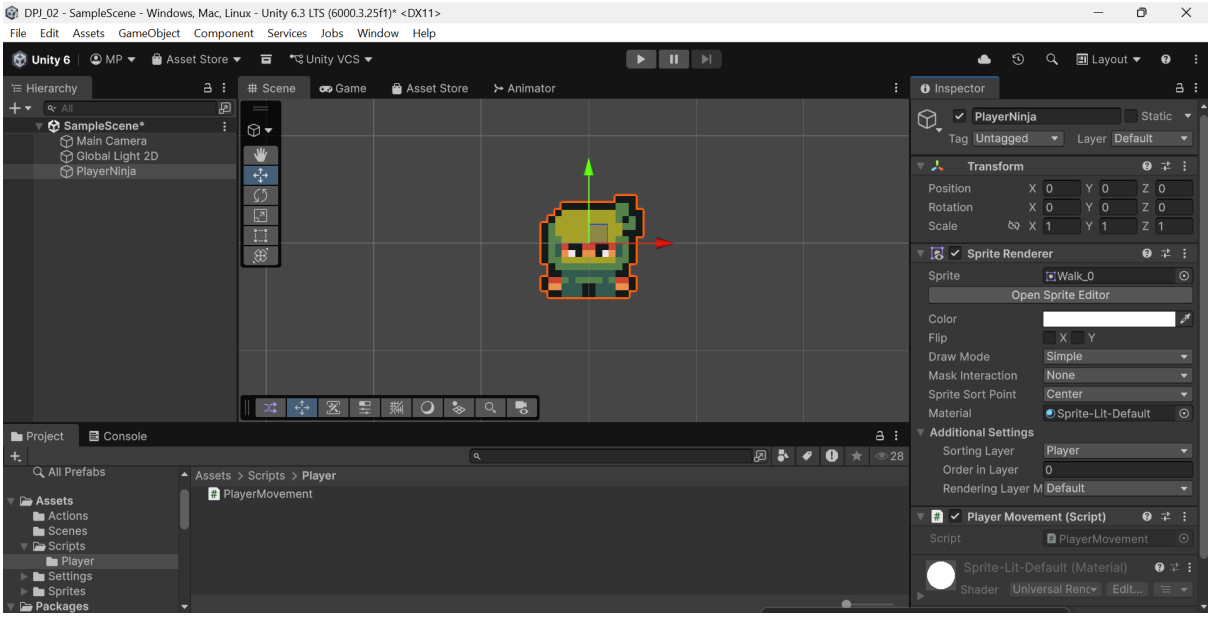


To start this level we searched for the “green ninja” animations selecting it and changing it to 16 pixels per unit, filter mode:point, compression:none and applied the changes.



Here we are at the sprite editor and we sliced it using the “grid by cell size” option, with pixel size on 16 pixels for x and y then applied.

Level 3



Level 4

On Level 4 we started creating the variables for the class created in Level 3, beginning with the following two seen in the image below:

```
Script de Unity | 0 referências
public class PlayerMovement : MonoBehaviour
{
    [Header("Config")]
    [SerializeField] private float speed;
```

To give access to the “Input Actions” created in Level 1 we add the following code in the awake method, while also creating the variable “actions” in the class.

```
Mensagem do Unity | 0 referências
private void Awake()
{
    actions = new InputActions();
}
```

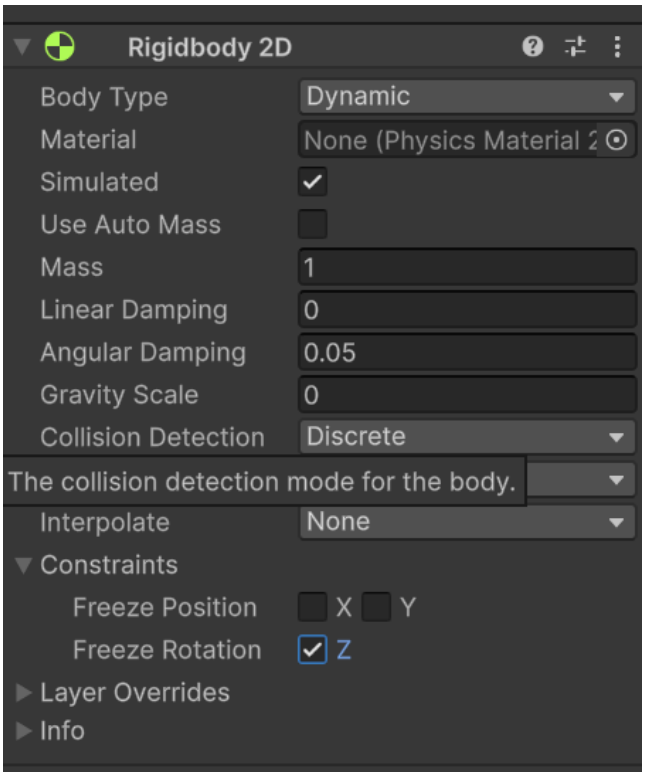
Then we added two new methods to the code, these are “OnEnable” and “OnDisable”, allowing us to disable and enable the “InputActions”.

```
Mensagem do Unity | 0 referências
private void OnEnable()
{
    actions.Enable();
}

Mensagem do Unity | 0 referências
private void OnDisable()
{
    actions.Disable();
}
```

Level 5

On Level 5 we finally implemented movement into the ninja, we started by adding a “Rigidbody2D” component to “PlayerNinja”, altered the gravity scale to 0, and stopped rotation on the Z axis.



We then need to initialize the component in the “PlayerMovement” script, similarly to “InputActions” we need to create a variable and write code in the awake method, but in awake we need to use “GetComponent” instead.

```
private Rigidbody2D rigidbody2d;

Mensagem do Unity | 0 referências
private void Awake()
{
    actions = new InputActions();
    rigidbody2d = new Rigidbody2D();
}
```

```
Mensagem do Unity | 0 referências
private void Awake()
{
    actions = new InputActions();
    rigidBody2d = GetComponent<Rigidbody2D>();
}
```

After that we created the “moveDirection” variable and three more methods:

```
1 referência
private void ReadMovement()
{
    moveDirection = actions.Movement.Move.ReadValue<Vector2>().normalized;
}

1 referência
private void Move()
{
    rigidBody2d.MovePosition(rigidBody2d.position + moveDirection * (speed * Time.fixedDeltaTime));
}

Mensagem do Unity | 0 referências
private void FixedUpdate()
{
    Move();
}
```

“ReadMovement” reads the input of the player, then “Move” moves tells the ninja to move, the player then moves in a determined frame time using the “FixedUpdate” method.

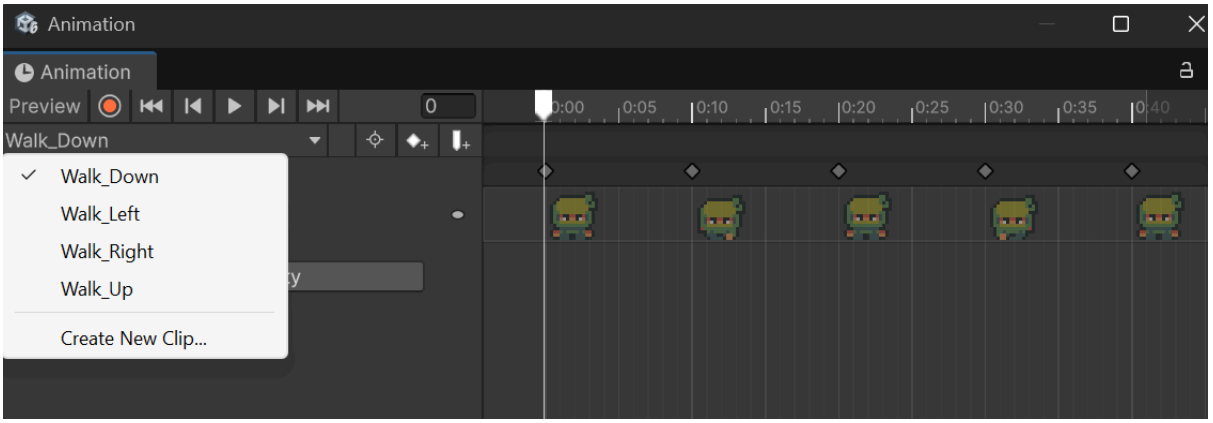
But on every single frame the movement is being read because it’s being called on the update method.

```
// Update is called once per frame
Mensagem do Unity | 0 referências
void Update()
{
    ReadMovement();
}
```

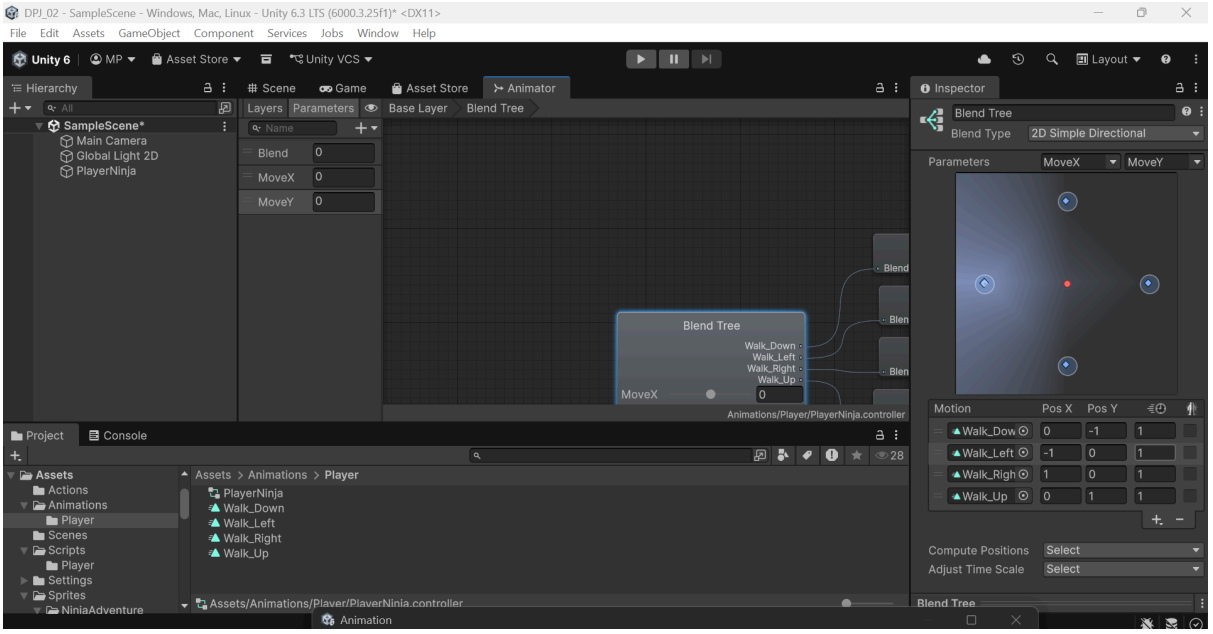
We can then alter the ninja’s speed on the inspect menu.



Next we needed to create animations using the animation window opened at the start of the project. We created four animations, one for each direction, by using the correct sprites on determined frames.



Opening the animator tab we verified that some animation knots were created, as asked, we deleted them and created a blend tree. In the blend tree we created two parameters, “MoveX” and “MoveY”, then we altered the blend type to “2D simple directional”, making sure that the parameters in the blend type were the two we just created. We then added four motions, so we dragged the animation files created in the animations>player folder to the according direction. For each motion we wrote the correct direction the player has to go by looking at it as the X and Y axis, for example, for the player to go down it goes “-1” on the Y axis (Vertical). To finish this part we made sure that the MoveX and MoveY values were set to 0 by default



In order to use the animations we had to edit the code again, so we created a private variable called animator, and initialised in it awake using “GetComponent”

```
private void Awake()
{
    animator = GetComponent<Animator>();
    actions = new InputActions();
    rigidBody2d = GetComponent<Rigidbody2D>();
}
```

To make sure it always works properly we added the following “if” statement in the “ReadMovement” method. Below it we added the code seen in the image, making sure all the names are the same as in the project.

```
private void ReadMovement()
{
    moveDirection = actions.Movement.Move.ReadValue<Vector2>().normalized;

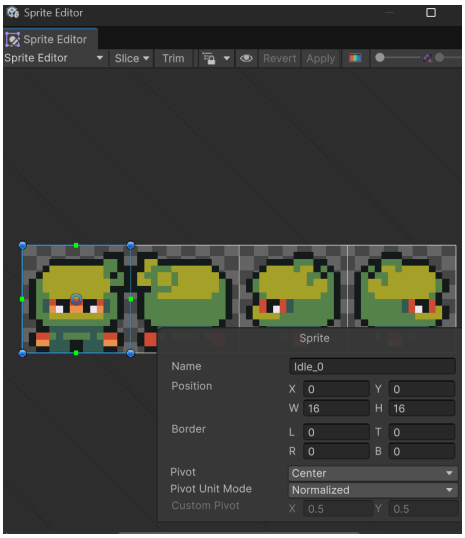
    if(moveDirection == Vector2.zero)
    {
        return;
    }
    animator.SetFloat("MoveX", moveDirection.x);
    animator.SetFloat("MoveY", moveDirection.y);

}
```

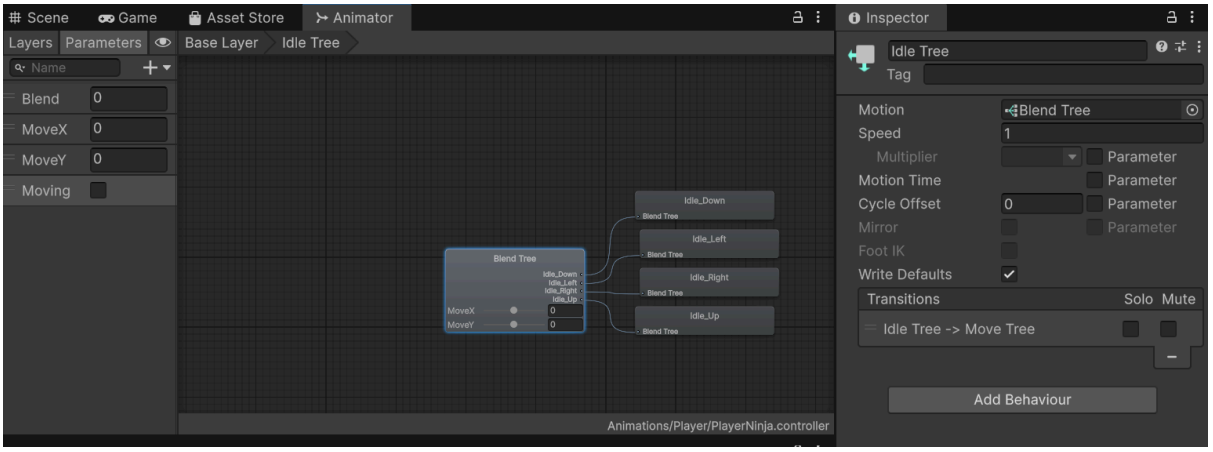
So we finished Level 5 and tested the project, the movement and animations were all working perfectly, but the ninja kept moving when idle, so we had to do the bonus level.

Bonus Level

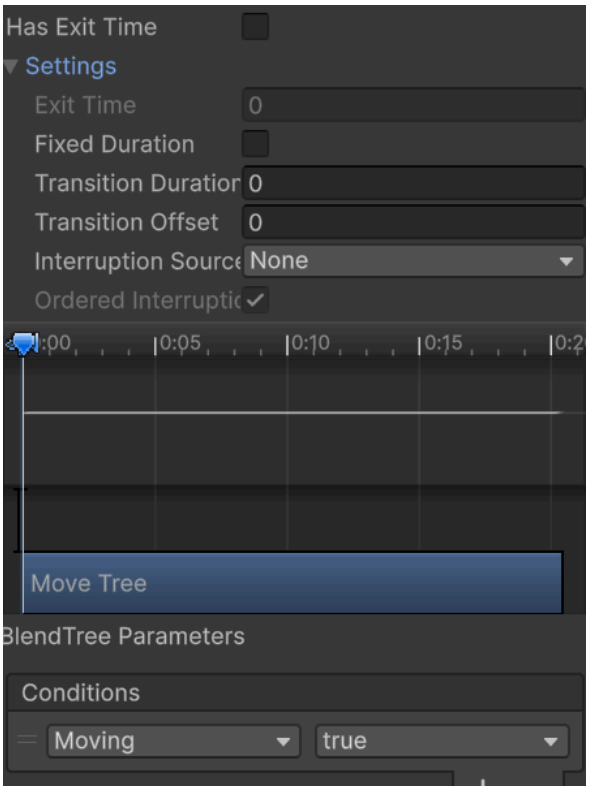
We did the same sprite cutting as the one in Level 3, but for the idle sprites.



Then made a new blend tree in the animator with the idle animations in each motion, making sure we also add a bool parameter called moving.



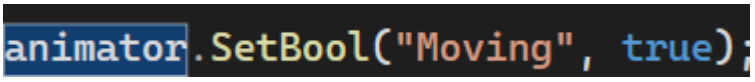
In the previous layer we made sure that entry was pointing to the new blend tree called “IdleTree” and made a transition from this tree to the previous “MoveTree”, this transition needed “Has exit time” and “Fixed Duration” deselected and “Transition Duration” set to 0. On the conditions tab we also added a new condition called “Moving” and set it to true.



To finish the project we only needed to add the following lines of code to the “PlayerMovement” script again.

```
animator.SetBool("Moving", false);
```

This one before the return in the “if” statement;



And this one right after the same statement.

When testing the project we saw no issues, the player moved correctly and the animations were activated on the correct inputs.